PERFORMANCE

HelixCode Notification Performance Guide

Overview

This document provides comprehensive guidance on performance testing, benchmarking, and optimization of the HelixCode notification system.

Performance Testing Categories

1. Benchmarks - Micro-level Performance

Benchmarks measure the performance of individual components and operations.

Location: `internal/notification/benchmark_test.go`, `internal/event/benchmark_test.go`

Purpose: - Measure individual function performance - Track performance regressions - Compare different implementations - Identify optimization opportunities

2. Load Tests - Realistic Production Scenarios

Load tests simulate realistic production workloads.

Location: `test/load/notification_load_test.go`

Purpose: - Validate system behavior under expected load - Measure throughput and latency - Test concurrent operations - Verify resource efficiency

3. Stress Tests - System Limits

Stress tests push the system beyond normal operating conditions.

Purpose: - Find breaking points - Test error handling under extreme load - Validate recovery mechanisms - Identify resource leaks

---

Running Performance Tests

Benchmarks

Run all benchmarks:

```
go test ./internal/notification -bench=. -benchmem -run=^$
go test ./internal/event -bench=. -benchmem -run=^$
```

Run specific benchmark:

```
go test ./internal/notification
        -bench=BenchmarkEngine_SendDirect -benchmem -run=^$
```

Run with custom iterations:

```
go test ./internal/notification -bench=BenchmarkQueue
        -benchtime=10s -run=^$
```

Save benchmark results:

```
go test ./internal/notification -bench=. -benchmem -run=^$ >
        bench_old.txt
# Make changes
go test ./internal/notification -bench=. -benchmem -run=^$ >
        bench_new.txt
benchcmp bench_old.txt bench_new.txt
```

Run with CPU profiling:

```
go test ./internal/notification -bench=BenchmarkEngine
        -cpuprofile=cpu.prof -run=^$
go tool pprof cpu.prof
```

Run with memory profiling:

```
go test ./internal/notification -bench=BenchmarkEngine
        -memprofile=mem.prof -run=^$
go tool pprof mem.prof
```

Load Tests

Run all load tests:

```
go test ./test/load -v
```

Run specific load test:

```
go test ./test/load -v -run=TestLoad_1000NotificationsPerSecond
```

Skip load tests (short mode):

```
go test ./test/load -v -short
```

Run with timeout:

```
go test ./test/load -v -timeout=30m
```

Stress Tests

Run stress tests:

```
go test ./test/load -v -run=TestStress
```

Benchmark Reference

Notification System Benchmarks

BenchmarkSlackChannel_Send

- **Measures:** Slack channel send performance
- **Typical:** ~190µs/op, ~9600 B/op, ~140 allocs/op

BenchmarkDiscordChannel_Send

- **Measures:** Discord channel send performance
- **Typical:** ~190µs/op, ~9300 B/op, ~130 allocs/op

BenchmarkNotificationEngine_RegisterChannel

- **Measures:** Channel registration overhead
- **Typical:** ~160ns/op, ~176 B/op, ~6 allocs/op

BenchmarkNotificationEngine_SendDirect

- **Measures:** Direct send performance (bypassing rules)
- **Typical:** <1µs/op with mock channel

BenchmarkNotificationEngine_SendNotificationWithRules

- **Measures:** Send with rule evaluation
- **Typical:** Depends on number of rules

BenchmarkRetryableChannel_SendSuccess

- **Measures:** Retry wrapper overhead when no retries needed
- **Typical:** <1µs/op

BenchmarkRetryableChannel_SendWithRetries

- **Measures:** Performance with actual retries
- **Typical:** Depends on backoff configuration

BenchmarkRateLimiter_Allow

- **Measures:** Rate limiter check performance
- **Typical:** <100ns/op

BenchmarkRateLimitedChannel_Send

- **Measures:** Rate-limited send performance
- **Typical:** <1µs/op when within limits

BenchmarkNotificationQueue_Enqueue

- **Measures:** Queue enqueue performance
- **Typical:** ~100ns/op

BenchmarkNotificationQueue_Dequeue

- **Measures:** Queue dequeue performance
- **Typical:** ~50ns/op

BenchmarkNotificationQueue_Throughput

- **Measures:** End-to-end queue throughput with workers
- **Typical:** Depends on worker count and channel speed

BenchmarkMetrics_RecordSent

- **Measures:** Metrics recording overhead
- **Typical:** ~50ns/op

BenchmarkEngine_ConcurrentSends

- **Measures:** Concurrent send performance
- **Typical:** Scalability with goroutines

BenchmarkMetrics_ConcurrentWrites

- **Measures:** Metrics thread safety overhead
- **Typical:** Lock contention impact

****BenchmarkQueue_Parallel_*Workers****

- **Measures:** Queue throughput with different worker counts
- **Comparison:** 1, 5, 10, 20 workers

Event Bus Benchmarks

BenchmarkEventBus_PublishSync

- **Measures:** Synchronous event publishing
- **Typical:** <1µs/op with simple handlers

BenchmarkEventBus_PublishAsync

- **Measures:** Asynchronous event publishing
- **Typical:** Lower latency than sync

BenchmarkEventBus_ConcurrentPublish

- **Measures:** Concurrent event publishing
- **Typical:** Scalability validation

BenchmarkEventBus_PublishMultipleSubscribers

- **Measures:** Performance with many subscribers
- **Typical:** Linear degradation with subscribers

Load Test Reference

TestLoad_1000NotificationsPerSecond

Scenario: 1000 notifications/second for 10 seconds

Parameters: - Duration: 10 seconds - Target Rate: 1000/sec - Queue Workers: 10 - Queue Size: 10000

Success Criteria: - Actual rate within 10% of target (900-1100/sec) - Failure rate < 1% - Queue drains within 30 seconds

Metrics: - Total sent - Total failed - Actual rate achieved - Final queue size

TestLoad_ConcurrentChannels

Scenario: 20 concurrent senders across 10 channels

Parameters: - Duration: 5 seconds - Concurrent Senders: 20 - Channels: 10 - Send Interval: 10ms

Success Criteria: - No errors - Even distribution across channels - Success rate > 99%

Metrics: - Total sent - Total failed - Rate per second - Success rate

TestLoad_QueueSaturation

Scenario: Saturate queue with slow channel processing

Parameters: - Queue Size: 100 - Workers: 5 - Channel Delay: 50ms - Rapid Fire: 200 notifications

Success Criteria: - Queue handles saturation gracefully - No notifications lost - Backpressure works correctly

Metrics: - Enqueued count - Rejected count - Processed count - Final queue state

TestLoad_RetryStorm

Scenario: High retry rate with flaky channels

Parameters: - Goroutines: 20 - Notifications per Goroutine: 50 - Failure Rate: 50% - Max Retries: 3

Success Criteria: - Success rate > 50% (with retries) - No deadlocks - Reasonable completion time

Metrics: - Total succeeded - Total failed - Success rate - Average attempts per notification

TestLoad_RateLimiterStress

Scenario: Attempt to exceed rate limits

Parameters: - Rate Limit: 100/second - Attempted: 500 notifications - Burst pattern

Success Criteria: - Rate stays below limit - No rate limit bypass - Smooth throttling

Metrics: - Attempted sends - Actual sends - Actual rate - Rate limit enforcement

TestLoad_EventBusHighVolume

Scenario: High-volume event publishing

Parameters: - Events: 10,000 - Subscribers: 5 - Async Mode: true - Processing Delay: 1ms

Success Criteria: - All events delivered - All handlers called - Completes within timeout

Metrics: - Published count - Processed count - Publish rate - Processing completion

TestLoad_MetricsUnderLoad

Scenario: Concurrent metric recording

Parameters: - Goroutines: 50 - Records per Goroutine: 1000 - Channels: 10

Success Criteria: - Accurate counts - No data races - Fast recording

Metrics: - Total records - Recording rate - Accuracy verification - Success rate calculation

---

Performance Optimization Guide

Channel Optimization

1. Batch API Requests

```
// Instead of single sends:
for _, notif := range notifications {
    channel.Send(ctx, notif)
}

// Batch when supported:
channel.SendBatch(ctx, notifications)
```

2. Connection Pooling

```
// Reuse HTTP clients
var httpClient = &http.Client{
    Timeout: 30 * time.Second,
    Transport: &http.Transport{
```

```
        MaxIdleConns:        100,
        MaxIdleConnsPerHost: 10,
        IdleConnTimeout:     90 * time.Second,
    },
}
```

3. Optimize Payload Size

```
// Minimize unnecessary data
notification := &Notification{
    Title:   "Alert",
    Message: summary,  // Not full logs
    Metadata: map[string]interface{}{
        "id": taskID,  // Just reference
    },
}
```

Queue Optimization

1. Right-Size Workers

```
// CPU-bound channels: workers = CPU cores
queue := NewNotificationQueue(engine, runtime.NumCPU(), 10000)

// I/O-bound channels: more workers
queue := NewNotificationQueue(engine, 20, 10000)
```

2. Adjust Queue Size

```
// Low latency: small queue
queue := NewNotificationQueue(engine, 10, 100)

// High throughput: large queue
queue := NewNotificationQueue(engine, 20, 10000)

// Memory-constrained: bounded queue with backpressure
queue := NewNotificationQueue(engine, 5, 500)
```

3. Priority Queuing (Future Enhancement)

```
// Separate queues by priority
urgentQueue := NewNotificationQueue(engine, 10, 1000)
normalQueue := NewNotificationQueue(engine, 5, 5000)
```

Rate Limiting Optimization

1. Channel-Specific Limits

```
// Use realistic limits per channel
limits := map[string]*RateLimiter{
    "slack":    NewRateLimiter(1, 1*time.Second),
    "discord":  NewRateLimiter(5, 5*time.Second),
    "telegram": NewRateLimiter(30, 1*time.Second),
    "email":    NewRateLimiter(10, 1*time.Minute),
}
```

2. Adaptive Rate Limiting (Future Enhancement)

```
// Adjust based on API responses
if resp.StatusCode == 429 {
    limiter.Reduce()
}
```

Retry Optimization

1. Exponential Backoff

```
config := RetryConfig{
    MaxRetries:     3,
    InitialBackoff: 1 * time.Second,
    MaxBackoff:     60 * time.Second,
    BackoffFactor:  2.0,  // Exponential
}
```

2. Circuit Breaker (Future Enhancement)

```
// Stop retrying if channel consistently fails
if failureRate > 0.8 {
    circuitBreaker.Open()
}
```

Event Bus Optimization

1. Async Mode for High Volume

```
// Sync for low-latency requirements
bus := NewEventBus(false)

// Async for high throughput
bus := NewEventBus(true)
```

2. Selective Subscription

```
// Only subscribe to needed events
bus.Subscribe(EventTaskFailed, handler)

// Not all events:
// bus.SubscribeAll(handler)  // Avoid
```

Metrics Optimization

1. Sampling (Future Enhancement)

```
// Sample metrics for very high volume
if rand.Float64() < 0.1 {  // 10% sampling
    metrics.RecordSent(channel, duration)
}
```

2. Aggregation

```
// Aggregate before recording
type aggregator struct {
    count    int
    totalTime time.Duration
}

func (a *aggregator) flush() {
    metrics.RecordSent(channel, a.totalTime/
        time.Duration(a.count))
}
```

Performance Targets

Throughput Targets

Notification Engine: - Single channel direct send: > 10,000/sec - With retry wrapper: > 5,000/sec - With rate limiting: Matches limit (e.g., 100/sec for Slack) - Queue throughput: > 1,000/sec with 10 workers

Event Bus: - Sync mode: > 100,000/sec - Async mode: > 500,000/sec - With 10 subscribers: > 10,000/sec

Latency Targets

Notification Operations: - Channel registration: < 1µs - Direct send (mock): < 1µs - Rule evaluation: < 100ns per rule - Queue enqueue: < 200ns - Queue dequeue: < 100ns

Event Operations: - Event publish (sync): < 10µs - Event publish (async): < 1µs - Subscription: < 1µs

Resource Targets

Memory: - Notification engine: < 1 MB base - Queue (1000 items): < 2 MB - Event bus: < 500 KB - Metrics: < 1 MB per 100,000 records

CPU: - Idle overhead: < 0.1% - Under load (1000/sec): < 10% - Worker goroutines: Minimal CPU when idle

Goroutines: - Base system: < 20 goroutines - Queue workers: 1 per worker - Event bus async: 1 per event (short-lived)

Profiling Guide

CPU Profiling

1. Generate CPU Profile

```
go test ./internal/notification -bench=BenchmarkQueue_Throughput
        \
    -cpuprofile=cpu.prof -run=^$
```

2. Analyze Profile

```
go tool pprof cpu.prof

# Interactive commands:
(pprof) top10           # Top 10 functions
(pprof) list SendDirect # Source code view
(pprof) web             # Visual graph (requires graphviz)
```

3. Web Interface

```
go tool pprof -http=:8080 cpu.prof
# Open http://localhost:8080 in browser
```

Memory Profiling

1. Generate Memory Profile

```
go test ./internal/notification -bench=BenchmarkEngine \
    -memprofile=mem.prof -run=^$
```

2. Analyze Allocations

```
go tool pprof mem.prof

(pprof) top10              # Top allocators
(pprof) list NewNotification
(pprof) alloc_space        # Switch to alloc space
```

3. Find Memory Leaks

```
# Run long-running test with memory profile
go test -memprofile=mem.prof -timeout=10m

# Look for growing allocations
go tool pprof -alloc_space mem.prof
```

Mutex Profiling

1. Generate Mutex Profile

```
go test ./internal/notification
        -bench=BenchmarkEngine_Concurrent \
    -mutexprofile=mutex.prof -run=^$
```

2. Find Contention

```
go tool pprof mutex.prof

(pprof) top10  # Most contended mutexes
(pprof) list   # Source locations
```

Block Profiling

1. Enable Block Profiling in Code

```
import "runtime"

func init() {
    runtime.SetBlockProfileRate(1)
}
```

2. Run with Block Profile

```
go test -blockprofile=block.prof
go tool pprof block.prof
```

Monitoring in Production

Key Metrics to Monitor

1. Throughput Metrics - Notifications sent per second - Notifications failed per second - Events published per second

2. Latency Metrics - Average send time per channel - p50, p95, p99 latencies - Queue wait time

3. Error Metrics - Failure rate per channel - Retry rate - Rate limit hits

4. Resource Metrics - Queue size - Active goroutines - Memory usage - CPU usage

Prometheus Integration (Future Enhancement)

```
import "github.com/prometheus/client_golang/prometheus"

var (
    notificationsSent = prometheus.NewCounterVec(
        prometheus.CounterOpts{
            Name: "notifications_sent_total",
            Help: "Total notifications sent",
        },
        []string{"channel", "type"},
    )

    sendDuration = prometheus.NewHistogramVec(
        prometheus.HistogramOpts{
            Name:    "notification_send_duration_seconds",
            Help:    "Notification send duration",
            Buckets: prometheus.DefBuckets,
        },
        []string{"channel"},
    )
)
```

Alert Rules

High Failure Rate:

```
alert: HighNotificationFailureRate
expr: rate(notifications_failed_total[5m]) > 0.1
for: 5m
severity: warning
```

Queue Saturation:

```
alert: NotificationQueueFull
expr: notification_queue_size > notification_queue_max * 0.9
for: 2m
severity: warning
```

Slow Channel:

```
alert: SlowNotificationChannel
expr: histogram_quantile(0.95,
        notification_send_duration_seconds) > 5
for: 10m
severity: warning
```

Troubleshooting Performance Issues

Issue: Low Throughput

Symptoms: - Actual rate far below target - Queue growing continuously - High latency

Diagnosis:

```
# Check CPU usage
top -p $(pgrep helixcode)

# Profile CPU
go test -bench=BenchmarkQueue_Throughput -cpuprofile=cpu.prof
go tool pprof cpu.prof
```

Solutions: 1. Increase worker count 2. Optimize channel implementations 3. Enable async event bus 4. Review rate limits

Issue: High Memory Usage

Symptoms: - Memory growing over time - OOM errors - GC pressure

Diagnosis:

```
# Memory profile
go test -memprofile=mem.prof
go tool pprof -alloc_space mem.prof

# Check for leaks
pprof> top10
pprof> list suspect_function
```

Solutions: 1. Reduce queue size 2. Clear old metrics 3. Fix goroutine leaks 4. Optimize payload sizes

Issue: Lock Contention

Symptoms: - High CPU usage - Low throughput - Slow response times

Diagnosis:

```
# Mutex profile
go test -mutexprofile=mutex.prof
go tool pprof mutex.prof
```

Solutions: 1. Reduce lock scope 2. Use read locks where possible 3. Shard data structures 4. Lock-free algorithms

Issue: Goroutine Leaks

Symptoms: - Growing goroutine count - Memory leaks - Eventual crashes

Diagnosis:

```
# Check goroutine count
curl http://localhost:6060/debug/pprof/goroutine

# Runtime stack traces
kill -QUIT $(pgrep helixcode)
```

Solutions: 1. Ensure all goroutines have exit conditions 2. Use context cancellation 3. Close channels properly 4. Review defer statements

---

Best Practices

1. Always Benchmark Changes

```
# Before changes
go test -bench=. > bench_before.txt

# After changes
go test -bench=. > bench_after.txt

# Compare
benchcmp bench_before.txt bench_after.txt
```

2. Profile in Production-Like Environment

- Use realistic data sizes
- Enable all middleware (retry, rate limit)
- Test with actual API endpoints
- Monitor resource usage

3. Set Performance Budgets

```
func TestPerformanceBudget(t *testing.T) {
    start := time.Now()

    // Operation under test
    err := engine.SendDirect(ctx, notif, []string{"mock"})

    elapsed := time.Since(start)

    // Budget: Must complete in 1ms
    if elapsed > 1*time.Millisecond {
        t.Errorf("Operation too slow: %v (budget: 1ms)", elapsed)
    }
}
```

4. Use Realistic Test Data

```
// Production-like notification
notif := &Notification{
    Title:   strings.Repeat("Long Title ", 10),
    Message: strings.Repeat("Long message content ", 100),
    Metadata: make(map[string]interface{}, 50),
}
```

5. Test Concurrent Scenarios

```go
func TestConcurrentLoad(t *testing.T) {
    var wg sync.WaitGroup
    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            engine.SendDirect(ctx, notif, channels)
        }()
    }
    wg.Wait()
}
```

Further Reading

- Go Performance Best Practices
- Go Profiling Guide
- High Performance Go
- Testing Guide
- API Reference
- Configuration Reference